PHP REST API Server Documentation (Project1)

1. Introduction

This project implements a REST API using PHP, MySQL, and Nginx

- REST architecture
- Secure endpoints with Bearer tokens
- Works locally via Nginx + PHP-FPM

2. Install MySQL

1. Download the MySQL Installer
2. During setup:
    - Set root password.
3. Also Download MySQL Workbench
4. Launch **MySQL Workbench** → Connect as `root`.
5. Create the database:

```
CREATE DATABASE musicdb CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE musicdb;
```

3. Creating Database Tables

```
CREATE TABLE users (
id INT AUTO_INCREMENT PRIMARY KEY,
username VARCHAR(50) UNIQUE,
token VARCHAR(64),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE songs (
id INT AUTO_INCREMENT PRIMARY KEY,
title VARCHAR(255),
artist VARCHAR(255),
genre VARCHAR(100),
filename VARCHAR(255),
uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

4. Download PHP

Download PHP

extract to C:/ and add PHP to your PATH

Begin running php by entering the terminal in C:/php and running:

php-cgi -b 127.0.0.1:9000

5. Download Nginx

Dowload Nginx extract to C:/

 alt text

 alt text

#6. Configure your nginx.conf file

server {

listen 80;

server_name localhost;

root C:/project1;

```
index index.php index.html;

location / {
    try_files $uri $uri/ /index.php?$query_string;
}

location ~ \.php$ {
    fastcgi_pass 127.0.0.1:9000;
    fastcgi_index index.php;
    fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
    include fastcgi_params;
}
```

Small Note:

Common issues thrown include:

500: Interal Server (Typically routing)

413: Using Files larger than 1MB

CORS blocked: header corrections

CORS Configuration"

index.php should contain

```
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type, Authorization");
```

Endpoint 1:

fetch all songs:

curl http://localhost/songs

Response:

{"id":1,"title":"Song1","artist":"Artist1","genre":"Pop","filename":"song.mp3"}

Endpoint 2:

fetch song by id:

curl http://localhost/songs/(id)

Response:

{"id":1,"title":"Song1","artist":"Artist1","genre":"Pop","filename":"song.mp3"}

Endpoint 3:

upload new song with Bearer token

curl -X POST http://localhost/songs
-H "Authorization: Bearer 1234567890abcdef"
-F "title=New Song"
-F "artist=Me"
-F "genre=Pop"
-F "file=@song.mp3"

Response:
{"message":"Song uploaded successfully"}

Endpoint 4:

update song information with Bearer token

curl -X PUT http://localhost/songs/1
-H "Authorization: Bearer 1234567890abcdef"
-d "title=UpdatedSong&artist=NewArtist&genre=Rock"

Response:
{"message":"Song updated successfully"}

Endpoint 5:

delete a song

curl -X DELETE http://localhost/songs/1
-H "Authorization: Bearer 1234567890abcdef"

Response:
{"message":"Song deleted successfully"}

Endpoint 6:

download the song file

curl -O http://localhost/songs/1/download

Response:
a file download

Endpoint 7:

filter songs by artist

curl http://localhost/songs/artist/Artist1

Response:
{"id":1,"title":"Song1","artist":"Artist1","genre":"Pop","filename":"song1.mp3"}

Endpoint 8:

filter songs by genre

curl http://localhost/songs/genre/Pop

Response:
{"id":1,"title":"Song1","artist":"Artist1","genre":"Pop","filename":"song1.mp3"}

Endpoint 9:

add a comment to a song

curl -X POST http://localhost/songs/1/comment
-H "Content-Type: application/json"
-d '{"comment":"Great song!"}'

Response:
{"message":"Comment added successfully"}

Endpoint 10:

search song by title

curl http://localhost/songs/title/Song1

Response:
{"id":1,"title":"Song1","artist":"Artist1","genre":"Pop","filename":"song1.mp3"}

Testing:

test.html has been provided as webpage testing.